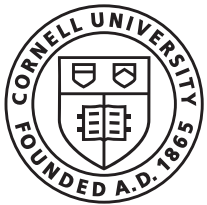


L4c: Shortest-Path Algorithms

CHEME 5800 Cornell University

Objectives for Today

Today we formulate the single-source shortest-path problem and compare two global schedules for the same local relaxation update.

Today's concepts

- Shortest paths and relaxation: calculate path cost and update distance and predecessor maps.
- Dijkstra's algorithm: explain greedy finalization under nonnegative edge weights.
- Bellman-Ford: explain repeated relaxation, negative edges, and reachable negative-cycle detection.

Lecture: Shortest-Path Algorithms

Lecture and Lab

Lecture: Shortest-Path Algorithms

Formulate the problem, trace Dijkstra and Bellman-Ford, and distinguish a permissible negative edge from a reachable negative-weight cycle.

Lab L4d: Production Planning with Shortest Paths

Predict a least-cost production route, compute it with Dijkstra, verify it independently with Bellman-Ford, and locate a cost threshold where the optimum changes.

Weighted Graphs Turn Routes into Optimization

An edge weight can represent travel time, communication delay, monetary cost, or the number of unit operations in a chemical process.

The common question is:

Among all feasible routes from a source to a target, which route has the smallest total edge weight?

Enumerating every route is usually impractical because the number of routes may grow exponentially with graph size.

A Feasible Route Follows Directed Edges

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be directed, let $w : \mathcal{E} \to \mathbb{R}$ assign edge weights, and fix source $s \in \mathcal{V}$. A feasible route from s to target t is:

$$P = \langle v_0, v_1, \ldots, v_k \rangle, \qquad v_0 = s, \quad v_k = t,$$

with $(v_i, v_{i+1}) \in \mathcal{E}$ for $i = 0, \ldots, k-1$.
Its total weight is the sum of its edge weights:

$$\underbrace{w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})}_{\text{route cost}} .$$

Distance Is Finite Only for a Reachable Target

Let $\mathcal{P}_{s,t}$ be all feasible routes from s to t. When no repeatable negative cycle makes the route cost unbounded below, define:

$$d(s,t) = \begin{cases} \min\limits_{P \in \mathcal{P}_{s,t}} w(P), & \mathcal{P}_{s,t} \neq \emptyset, \\ +\infty, & \mathcal{P}_{s,t} = \emptyset. \end{cases}$$

Thus $d(s,t) = +\infty$ means the target is unreachable. When the optimum is finite, a selected shortest path P^* satisfies:

$$P^* \in \arg\min_{P \in \mathcal{P}_{s,t}} w(P), \qquad w(P^*) = d(s,t).$$

A Single-Source Solution Returns Two Maps

Fix source s and compute, for every vertex $v \in \mathcal{V}$:

$$\underbrace{\mathrm{dist}[v] = d(s, v)}_{\text{minimum route cost}}, \qquad \underbrace{\mathrm{prev}[v]}_{\text{one predecessor on a selected route}} .$$

- $\mathrm{prev}[s] = \texttt{nothing}$.
- Follow predecessors backward from a target, then reverse the sequence, to reconstruct one minimizing route.
- If several routes tie, the predecessor map may encode any one of them.
- An unreachable vertex keeps distance $+\infty$ and no predecessor.

Shortest Paths Have Optimal Substructure

Suppose a shortest path from source s to target t passes through vertex u.

Its prefix from s to u must itself be a shortest path.

If a cheaper prefix existed, replacing the old prefix would produce a cheaper route to t, contradicting optimality.

A globally optimal route is assembled from optimal prefixes.

Algorithmic consequence: store the best discovered prefix to each vertex instead of enumerating complete routes.

Relaxation Starts with One Known Route

At initialization, the zero-edge route from s to itself is the only known route:

$$\mathrm{dist}[v] = \begin{cases} 0, & v = s, \\ +\infty, & v \neq s, \end{cases} \qquad \mathrm{prev}[v] = \texttt{nothing}.$$

A finite estimate $\mathrm{dist}[v]$ is always the weight of an actual source-to-v route discovered so far.

These estimates begin conservatively and can only decrease as better routes are found.

Relax One Extended Route

Consider edge $(u, v) \in \mathcal{E}$. If a finite route to u is known, appending the edge gives candidate cost:

$$\underbrace{\mathrm{alt}(u, v) = \mathrm{dist}[u] + w(u, v)}_{\text{candidate route to } v}.$$

Relaxation updates both maps only when the candidate is strictly cheaper:

$$(\mathrm{dist}[v], \mathrm{prev}[v]) \leftarrow \begin{cases} (\mathrm{alt}(u, v), u), & \mathrm{alt}(u, v) < \mathrm{dist}[v], \\ (\mathrm{dist}[v], \mathrm{prev}[v]), & \mathrm{alt}(u, v) \geq \mathrm{dist}[v]. \end{cases}$$

The strict comparison preserves the existing predecessor when costs tie.

Relaxation Preserves Two Useful Invariants

Every finite estimate is the weight of a discovered route, so it is an upper bound on the unknown optimum:

$$\underbrace{d(s, v) \leq \mathrm{dist}[v]}_{\text{upper-bound invariant}}.$$

Immediately after relaxing (u, v), the estimates also satisfy:

$$\underbrace{\mathrm{dist}[v] \leq \mathrm{dist}[u] + w(u, v)}_{\text{local consistency for } (u,v)}.$$

Repeated relaxation propagates cheaper prefixes. The global algorithm decides which edges are revisited and when an estimate becomes final.

Dijkstra and Bellman-Ford use the same relaxation rule. They differ in the order, repetition, and assumptions surrounding that update.

Dijkstra Requires Nonnegative Edge Weights

Dijkstra repeatedly selects the unsettled vertex with the smallest tentative distance. Its greedy step requires:

$$w(u, v) \geq 0 \qquad \text{for every } (u, v) \in \mathcal{E}.$$

Initialize:

- distance and predecessor maps;
- an empty settled set $\mathcal{S}$;
- a min-priority queue $\mathcal{Q}$ containing s with priority zero.

Every other distance begins at $+\infty$. The public implementation rejects a graph containing any negative edge before running the greedy algorithm.

Dijkstra Finalizes One Vertex at a Time

While the min-priority queue $\mathcal{Q}$ is not empty:

1. Remove a vertex u with minimum priority.
2. If $u \in \mathcal{S}$, skip it; otherwise add u to $\mathcal{S}$. Its distance is now final.
3. For every outgoing edge (u, v), compute $\mathrm{alt} = \mathrm{dist}[u] + w(u, v)$.
4. If $\mathrm{alt} < \mathrm{dist}[v]$, update the distance, set $\mathrm{prev}[v] = u$, and assign priority alt to v.

A weighted adjacency list supplies the outgoing target and weight pairs needed in step 3.

Why Is a Settled Distance Final?

Let u be the unsettled vertex with minimum tentative distance. Suppose a shorter route to u exists.

On that route, let y be the first unsettled vertex and x its settled predecessor. When x was settled, relaxing (x, y) produced a tentative distance no larger than the route prefix to y.

All remaining edge weights are nonnegative, so that prefix would be cheaper than $\mathrm{dist}[u]$. This contradicts the choice of u:

$$u \in \arg\min_{v \notin \mathcal{S}} \mathrm{dist}[v] \quad \Longrightarrow \quad \mathrm{dist}[u] = d(s, u).$$

Dijkstra Uses a Queue to Scan Edges

With a weighted adjacency list and a min-priority queue, Dijkstra runs in:

$$\underbrace{\mathcal{O}((|\mathcal{V}| + |\mathcal{E}|) \log |\mathcal{V}|)}_{\text{Dijkstra running time}}.$$

- Each reached vertex enters the queue as tentative distances improve.
- Each outgoing edge is examined when its source is settled.
- The settled set prevents a vertex from being finalized twice.

Scope: the bound matches the course implementation using adjacency lists and a comparison-based priority queue.

Bellman-Ford Repeatedly Relaxes Every Edge

Bellman-Ford permits negative edges by giving later improvements time to propagate across earlier vertices.

Initialize the same distance and predecessor maps. Then repeat at most $|\mathcal{V}| - 1$ times:

1. Visit every edge $(u, v) \in \mathcal{E}$.
2. If $\mathrm{dist}[u]$ is finite and the candidate is cheaper, relax (u, v).
3. If a complete pass changes nothing, stop early because the estimates converged.

No vertex is greedily settled. The algorithm instead expands the maximum path length whose optimal cost has propagated.

Pass k Accounts for Paths with at Most k Edges

After pass k, Bellman-Ford has accounted for every source-to-vertex path containing at most k edges.

If no reachable negative-weight cycle exists, a shortest path can be chosen to be simple. Repeating a vertex would add either a nonnegative cycle or a removable zero-cost cycle.

A simple path visits at most $|\mathcal{V}|$ vertices and therefore contains at most:

$$\underbrace{|\mathcal{V}| - 1}_{\text{maximum edges in a simple path}}$$

This is why $|\mathcal{V}| - 1$ relaxation passes are sufficient.

One Extra Pass Detects Negative Cycles

After the usual passes, scan every edge once more.

If an edge from a finite-distance source can still be relaxed, some reachable cycle has negative total weight. Repeating that cycle drives route costs downward:

$$w(P), \quad w(P) + w(C), \quad w(P) + 2w(C), \quad \ldots \longrightarrow -\infty \qquad \text{when } w(C) < 0.$$

No finite shortest-path solution exists for vertices affected by that cycle.

A negative cycle disconnected from source s does not invalidate the single-source result because its edges begin at infinite-distance vertices.

Bellman-Ford Trades Time for Flexibility

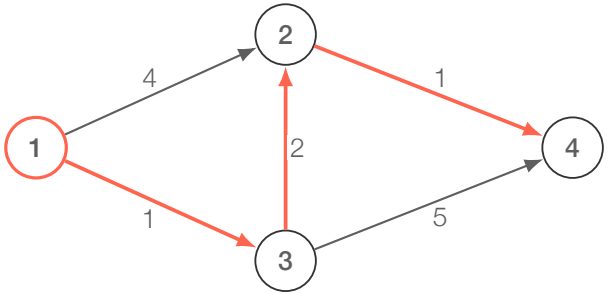
In the worst case, Bellman-Ford scans every edge during each of $|\mathcal{V}| - 1$ passes and during the cycle check:

$$\underbrace{\mathcal{O}(|\mathcal{V}|\,|\mathcal{E}|)}_{\text{running time}}, \qquad \underbrace{\mathcal{O}(|\mathcal{V}|)}_{\text{distance and predecessor storage}} .$$

Early exit can reduce the actual number of passes when a complete edge scan produces no update.

The benefit is correctness with negative edges and explicit detection of reachable negative-weight cycles.

Worked Comparison: A Nonnegative Graph



Both algorithms are valid because all weights are nonnegative. The minimum-cost route from 1 to 4 is:

$$1 \to 3 \to 2 \to 4, \qquad 1 + 2 + 1 = 4.$$

Dijkstra Trace on the Worked Graph

Step	Action	d_1	d_2	d_3	d_4
0	initialize at source 1	0	∞	∞	∞
1	settle 1; relax $(1,2)$ and $(1,3)$	0	4	1	∞
2	settle 3; improve 2 and discover 4	0	3	1	6
3	settle 2; improve 4	0	3	1	4
4	settle 4	0	3	1	4

The predecessor updates are $\mathrm{prev}[3] = 1$, $\mathrm{prev}[2] = 3$, and $\mathrm{prev}[4] = 2$. Following them backward reconstructs $4 \leftarrow 2 \leftarrow 3 \leftarrow 1$.

Different Schedules Produce the Same Optimum

On the nonnegative worked graph:

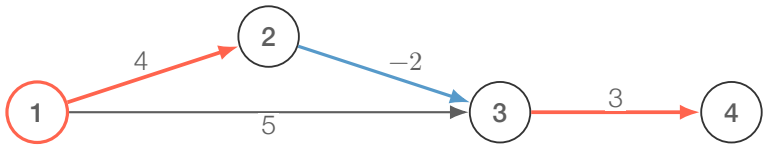
$$\mathrm{dist}_{\mathrm{Dijkstra}}[v] = \mathrm{dist}_{\mathrm{Bellman-Ford}}[v] \qquad \text{for every vertex } v.$$

Both predecessor maps reconstruct the unique target route:

$$P^* = \langle 1, 3, 2, 4 \rangle, \qquad w(P^*) = 4.$$

Agreement is expected because Dijkstra's nonnegative-weight precondition holds. Bellman-Ford provides an independent check using a different relaxation schedule.

A Negative Edge Can Still Have a Finite Optimum



Bellman-Ford finds the finite route:

$$1 \to 2 \to 3 \to 4, \qquad 4 - 2 + 3 = 5.$$

A negative edge is permissible when no reachable negative cycle can be repeated.

Why Dijkstra Rejects the Negative-Edge Graph

Dijkstra may settle a vertex before a later negative edge reveals a cheaper route.

In the example, the direct tentative cost to vertex 3 is 5, but the route through vertex 2 has cost:

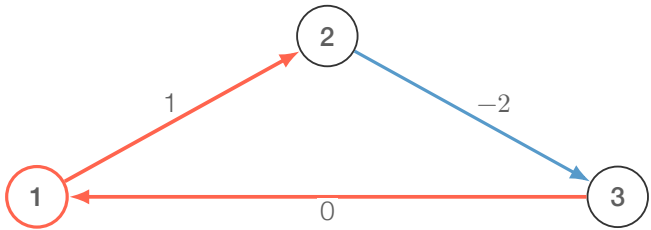
$$\mathrm{dist}[2] + w(2,3) = 4 + (-2) = 2.$$

Nonnegative weights guarantee that extending a path cannot lower its cost. The edge of weight -2 removes that guarantee, so the implication

$$\text{minimum tentative distance} \Longrightarrow \text{final distance}$$

is no longer valid. The implementation therefore rejects the graph explicitly.

Negative Cycles Remove Finite Optima



The cycle weight is:

$$w(C) = 1 - 2 + 0 = -1.$$

Each repetition lowers the route cost by one. Bellman-Ford's extra pass detects another possible relaxation and reports that no finite solution exists.

The Public Contracts Separate Three Cases

The notebook turns the comparison into executable checks:

- **Nonnegative graph:** both algorithms return $\langle 1, 3, 2, 4 \rangle$ with target cost 4 and identical distance maps.
- **Negative edge, no negative cycle:** Bellman-Ford returns $\langle 1, 2, 3, 4 \rangle$ with target cost 5.
- **Violated contract:** Dijkstra raises an error for the negative edge; Bellman-Ford raises an error for the reachable negative cycle.

Purpose: the tests verify numerical results and the public-interface assumptions, not only one displayed example.

Choose the Algorithm from the Weight Contract

Question	Dijkstra	Bellman-Ford
Edge weights	All nonnegative	Negative edges allowed
Global schedule	Settle minimum tentative vertex	Relax every edge repeatedly
Negative-cycle detection	Not supported	Detects a reachable cycle
Worst-case time	$\mathcal{O}((\lvert\mathcal{V}\rvert + \lvert\mathcal{E}\rvert) \log \lvert\mathcal{V}\rvert)$	$\mathcal{O}(\lvert\mathcal{V}\rvert\lvert\mathcal{E}\rvert)$
Natural storage	Weighted adjacency list	Edge list

Both return distance and predecessor maps when their input contracts hold.

Lab L4d Applies Both Algorithms

The lab maps alternative production routes to a weighted directed graph.

1. Predict the least-cost route by inspecting the process costs.
2. Compute the route with Dijkstra because all costs are nonnegative.
3. Verify the distance and route independently with Bellman-Ford.
4. Discount one production step and locate the break-even cost where the selected route changes.

Sensitivity analysis asks when the optimum changes, not only what the optimum is today.

Summary

A shortest-path algorithm combines a local relaxation rule with a global schedule for deciding which edges to revisit and when a distance becomes final.

Key takeaways

- **Relaxation connects local edges to global routes.** Distance estimates store costs; predecessors store route choices.
- **Dijkstra finalizes greedily.** Nonnegative edge weights make the minimum tentative distance safe to settle.
- **Bellman-Ford revisits every edge.** Repeated passes allow negative edges; one extra pass detects a reachable negative cycle.

In L4d, both algorithms validate a production route and reveal when a cost perturbation changes the optimum.

That's a wrap. Which property of the edge weights tells you which shortest-path algorithm is valid?